山东大学 计算机科学与技术 学院

数据结构与算法 课程实验报告

学号：202200130077	姓名：薛润泽	班级：人工智能
实验题目：实验二 排序算法		
实验学时：2	实验日期： 2023.9.20	
实验目的： 掌握各种简单排序算法。		
软件开发工具： DEV C++		

1. 实验内容

（1）、 创建排序类：数据含有 n 个整数，使用动态数组存储；提供操作：按名次排序、及时终止的选择排序、及时终止的冒泡排序、插入排序；

（2）、 键盘输入 n, 随机生成 n 个 0~1000 之间的整数建立排序实例；输出各种排序算法的排序过程。

（3）、 统计每一种排序所耗费的时间（即比较次数和移动次数）。

2. 数据结构与算法描述 （整体思路描述，所需要的数据结构与算法）

（1） 名次排序

名次排序的关键步骤是对每个元素计算比该元素小的元素个数以及该元素左边与该元素相等的个数。所以可以 new 一个新的 r[n]数组，并对其初始化为 0，用来记录对应索引位置的元素名次。使用双重 for 循环来进行名次的计算。最后按照名次进行排序。

（2） 及时终止的选择排序

选择排序的思想是每一次选出剩余序列中的最大的元素，并与该剩余序列的最后一位元素交换。当选择排序全部序列实际已经有序时，如果选择的元素没有选择到序列首，则还会进行选择排序。为了实现选择排序的终止，定义一个 bool 类型的标记 sorted，当剩余序列无序时将其置 0，有序时置 1，在循环条件中使用!sorted 来判断是否结束循环。

（3） 及时终止的冒泡排序

冒泡排序的思想是遍历全部序列，从左到右依次交换满足交换条件的两两元素。为了实现冒泡排序的及时终止，可以定义一个 bool 类型的标记 sorted，sorted 置 0 为有序，sorted 置 1 为无序，如果在排序中一次两两交换也没有发生，说明该序列已经有序，sorted 置 0，结束冒泡排序。

（4） 插入排序

插入排序的思想是将给定的元素插入到一个有序序列中，并使得最终的序列有序。可以从第 1 个元素看起，依次将第 2 个，第 3 个，第 n 个元素插入到原有的序列中。

3. 测试结果（测试输入，测试输出）

测试输入:

5

5 3 4 2 1

测试输出:

1 2 3 4 5

4. 分析与探讨（结果分析，若存在问题，探讨解决问题的途径）

结果符合预期，四种排序程序运行正常。

要注意的是，在及时终止的选择排序，及时终止的冒泡排序，要定义一个标志来判断序列是否已经有序。在插入排序中，可以在原有数组上进行元素插入，不过要注意每次插入时，元素个数不断增大，索引不断从 1 到 n，最终完成整个数组的排序。

名次排序中最坏比较次数为 1+2+3+……+n-1 = （n-1）*n/2
移动次数为 2n
选择排序中最坏比较次数为 n-1+n-2+……+1 =（n-1）*n/2
移动次数为 3(n-1)
冒泡排序中最好比较次数为 n-1，最坏比较次数为(n-1)*n/2
插入排序中最好比较次数为 n-1，最坏比较次数为(n-1)*n/2

5. 附录：实现源代码（本实验的全部源程序代码，程序风格清晰易理解，有充分的注释）

```
#include<iostream>
template<typename T>
void swap(T &a, T&b){
    T c = a;
    a = b;
    b = c;
}
template<typename T>
class Solution{
public:
    Solution(T n):n(n){//构造函数完成对 n 和 a 数组的赋值
        for(int i = 0; i < n; i++){
            int temp = 0; scanf("%d",&temp);
            a[i] = temp;
        }
    };
    void RankSort();          //名次排序
    void SelectionSort();        //选择排序
    bool bubble(int n);
    void BubbleSort();           //冒泡排序
    void InsertSort();           //插入排序
    void insert(int n, T x);
    void OutAns();               //输出答案
    ~Solution(){
        delete []a;
    }
private:
    int n;
    T *a = new T[n];
};
```

```
template<typename T>
void Solution<T>::RankSort(){
    int *r = new int [n];                   //rank 数组用来计数
    for(int i = 0; i < n; i++){
        r[i] = 0;                           //初始化 rank 数组
    }
    for(int i =0 ; i < n; i++){
        for(int k = 0; k < i; k++){
            if(a[i] >= a[k]) r[i]++;        //计算名次,所有比 a[i]小的或者在 a[i]左边
与本元素相等的，都让 r[i]++
            else r[k]++;                    //否则 r[k]++
        }
    }
    T *u = new T[n];
    for(int i = 0; i < n; i++){
        u[r[i]] = a[i];
    }
    for(int i = 0; i < n; i++){
        a[i] = u[i];
    }
    delete []r; delete []u;
}
template<typename T>
void Solution<T>::OutAns(){
    for(int i = 0; i < n; i++){
        std::cout<<a[i]<<' ';
    }
}
template<typename T>
void Solution<T>::SelectionSort(){          // 选择排序实现,
    bool sorted = 0;
    for(int i = n-1; !sorted&&i >= 0; i--){
            sorted = 1;
            int indexOfMax = 0;
        for(int j = 0; j <= i; j++){
            if(a[j] >= a[indexOfMax]) {     //选择出更大的元素
                indexOfMax = j;
            }
            else{
                sorted = 0;                 //如果出现一次比 a[indexOfMax]要小的
元素，因为 a[indexOfMax]在 a[j]的左侧，则势必再要进行一次排序
            }
        }
        std::swap(a[i],a[indexOfMax]);
    }
}
```

```
template<typename T>
bool Solution<T>::bubble(int n){          //冒泡排序实现，bubble 只遍历所有元素一次，并将最大元素带到数组末尾
       bool sorted = 0 ;
       for(int j = 0; j < n-1; j++){
           if(a[j]>a[j + 1]) {
           swap(a[j],a[j + 1]);
           sorted = 1;
       }
   }
   return sorted;                         //如果 sorted ==0 说明一次交换都没有发生，说明所有元素有序
}
template<typename T>
void Solution<T>::BubbleSort(){
   for(int i = n; i > 1&&bubble(i); i--);  //及时停止的冒泡排序实现
}
template<typename T>
void Solution<T>::insert(int n,T x){      //插入排序中,给定一个数值,将该数值插入到已经有序的数组中的方法
   T temp = x;
   int i;
   for(i = n - 1; i >= 0 && a[i] > x; i --){
       a[i + 1] = a[i];                  //给定数值比第 i 位小，则将第 i 位向后移一位
   }
   a[i + 1]  =  x;
}
template<typename T>
void Solution<T>::InsertSort(){
   for(int i = 0; i < n; i++){           //从第一元素开始进行排序
       insert(i, a[i]);
   }
}
int main(){
   int n = 0;
   scanf("%d",&n);
   Solution<int> s(n);
   s.BubbleSort();
   s.OutAns();
   return 0;
}
```